

Lista de Nivelamento Python

Treinamento Intensivo de Sintaxe e Fundamentos — Backend Django

1. Tipos de Dados e Conversões

MUITO FÁCIL beecrowd 1003 — Soma Simples

Receba dois inteiros, calcule a soma algébrica entre eles e retorne o valor resultante.

```
Assinatura: def soma(a: int, b: int) -> int
Exemplo: soma(30, 10) == 40
```

FÁCIL Sanitizador de Parâmetro Numérico

Receba uma string com espaços ou quebras de linha e retorne o valor numérico limpo convertido para inteiro.

```
Assinatura: def sanitizar_parametro(param: str) -> int
Exemplo: sanitizar_parametro(" 42 \n") == 42
```

MÉDIO Formatador de Moeda para API

Receba um valor de ponto flutuante e retorne uma string formatada com prefixo "R\$ " e duas casas decimais.

```
Assinatura: def formatar_moeda(valor: float) -> str
Exemplo: formatar_moeda(1250.5) == "R$ 1250.50"
```

2. Estruturas Condicionais

MUITO FÁCIL beecrowd 1044 — Múltiplos

Avalie se dois números inteiros são múltiplos entre si (ou seja, se a divisão de um pelo outro deixa resto 0).

```
Assinatura: def sao_multiplos(a: int, b: int) -> bool
Exemplo: sao_multiplos(6, 24) == True
```

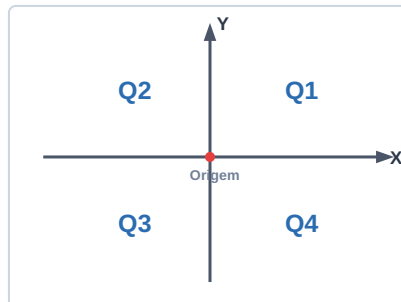
FÁCIL HackerRank — Ano Bissexto

Determine se um ano é bissexto: divisível por 4, exceto múltiplos de 100 (a menos que sejam também múltiplos de 400).

```
Assinatura: def eh_bissexto(ano: int) -> bool
Exemplo: eh_bissexto(2000) == True
```

MÉDIO beecrowd 1041 — Coordenadas de um Ponto

Com base na divisão do plano cartesiano ilustrada abaixo, receba as coordenadas de ponto flutuante (x, y) e determine se o ponto está na "Origem", sobre um dos eixos ("Eixo X", "Eixo Y") ou em um dos quatro quadrantes ("Q1", "Q2", "Q3", "Q4").



Assinatura: `def localizar_ponto(x: float, y: float) -> str`

Exemplo: `localizar_ponto(0.1, 0.1) == "Q1"`

3. Laços de Repetição

MUITO FÁCIL beecrowd 1067 — Números Ímpares

Retorne uma lista contendo todos os números inteiros ímpares no intervalo fechado de 1 até n.

Assinatura: `def obter_impares(n: int) -> list[int]`

Exemplo: `obter_impares(8) == [1, 3, 5, 7]`

FÁCIL LeetCode 412 — Fizz Buzz

Gere uma lista de 1 a n substituindo múltiplos de 3 por "Fizz", de 5 por "Buzz" e de ambos por "FizzBuzz".

Assinatura: `def fizz_buzz(n: int) -> list[str]`

Exemplo: `fizz_buzz(5) == ["1", "2", "Fizz", "4", "Buzz"]`

MÉDIO beecrowd 1165 — Número Primo

Verifique se um inteiro n (> 1) é divisível unicamente por 1 e por ele mesmo, retornando o booleano correspondente.

Assinatura: `def eh_primo(n: int) -> bool`

Exemplo: `eh_primo(97) == True`

4. Tratamento de Exceções (try / except)

MUITO FÁCIL Divisão com Captura de Zero

Execute a divisão de a por b. Trate explicitamente `ZeroDivisionError` para retornar `None` em divisões inválidas.

```
Assinatura: def dividir(a: float, b: float) -> float | None
Exemplo: dividir(10.0, 0.0) is None
```

FÁCIL Conversor Seguro de Inteiro

Tente converter uma string para inteiro. Intercepte `ValueError` caso a conversão falhe e retorne o sentinela `-1`.

```
Assinatura: def parse_inteiro(texto: str) -> int
Exemplo: parse_inteiro("abc") == -1
```

MÉDIO Somador de Lista Heterogênea

Percorra uma lista mista somando valores conversíveis para `float`. Capture `ValueError` e `TypeError` para ignorar itens não numéricos.

```
Assinatura: def somar_validos(itens: list) -> float
Exemplo: somar_validos(["10.5", "erro", 5, None, "2.5"]) == 18.0
```

5. Estruturas de Dados

MUITO FÁCIL LeetCode 217 — Contains Duplicate

Verifique a presença de elementos repetidos em uma lista de inteiros utilizando um conjunto (`set`).

```
Assinatura: def contem_duplicata(nums: list[int]) -> bool
Exemplo: contem_duplicata([1, 2, 3, 1]) == True
```

FÁCIL beecrowd 1281 — Ida à Feira

Dado um dicionário de preços e uma lista de compras [(produto, quantidade)], calcule o custo financeiro total.

```
Assinatura: def calcular_feira(tabela: dict[str, float], compras: list[tuple[str, int]]) -> float
Exemplo: calcular_feira({"banana": 2.5, "maca": 4.0}, [("banana", 2), ("maca", 1)]) == 9.0
```

MÉDIO **LeetCode 1 — Two Sum**

Encontre os índices dos dois elementos cujo somatório atinge o valor alvo utilizando um dicionário para busca em tempo $O(n)$.

Assinatura: `def dois_soma(nums: list[int], alvo: int) -> tuple[int, int]`

Exemplo: `dois_soma([2, 7, 11, 15], 9) == (0, 1)`